

SlimPer: 让个性化模型更轻量、更智能

Siqi Wang*, Xianjie Chen*†, Shaofeng Deng, Albert Chen, Romil Shah, Jiawei Huang, Zhaoqin Wang, Zhang Zhang, Yiqun Liu, Meilei Jiang, Anish Dubey, Moyan Mei, Tongxin Wang, Nathan Berrebbi, Misael Manjarres, Armand Sauzay, Shardul Kothapalli, Aryaman Vinchhi, Kevin Johnstone, Juheon Lee, Gufan Yin, Ziheng Huang, Justin Lin, Mert Terzihan, Yilin Qi, Cynthia Yang, Colin Peppler, Qi Ding, Ruohan Sun, Ge Song, Litao Deng, Parichay Kapoor, Matt Ma, Huihui Cheng, Jiyuan Zhang, Yanli Zhao, Yiping Han, Fangqiu Han, Ning Yao, Arun Singh, Jordan Edwards, Zhengyu Su, Abhishek Kumar, Guangdeng Liao, Ankit Asthana

Meta Platforms, Inc.
USA

摘要

工业推荐系统日益采用 Transformer 风格架构, 但这类架构继承了一个与推荐任务并不匹配的设计前提: 生成模型依赖逐标记自回归预测, 因此需要维护随序列长度增长的大型中间张量。相比之下, 推荐系统只为每个 <用户, 物品> 对生成一组相关性分数, 并不存在标记级监督。基于这一观察, 我们提出 **SlimPer**, 将个性化排序重新表述为对紧凑、统一的 <用户, 物品> 知识库进行迭代精炼。在每一层中, 模型选择性查询原始多模态用户侧标记, 计算显式相关性匹配分数, 并精炼知识库; 整个过程使用固定大小的中间表示, 每层复杂度为 $\mathcal{O}(N)$ 。由此, 模型深度与用户历史长度实现解耦, 能够在计算量和内存不按比例增长的情况下加深对相关性的理解; 仅请求优化 (request-only optimization) 还通过让所有候选物品共享一份用户侧标记, 进一步降低内存开销。SlimPer 在统一骨干中融合稀疏、稠密和序列特征, 并通过注意力机制提供内在可解释性。SlimPer 已部署于 Instagram Reels 和 Feed, 在简化整体系统的同时带来可衡量的用户参与度提升, 并可有效建模超过一万个细粒度用户历史事件。

CCS Concepts

• 信息系统 → 推荐系统。

1 引言

1.1 Transformer 与推荐任务的不匹配

判别式个性化模型通常被称为排序模型, 它们决定了用户每天接触的内容信息流、视频推荐和商品建议。这类模型需要依据用户偏好实时排列候选物品, 同时还要在生产成本约束下保持准确的预测。

工业推荐模型通常接收多模态输入特征, 不同模态在计算成本和捕获细粒度信息的能力之间具有不同的权衡 (详细比较见表 1)。常见做法是由相互独立的专用组件处理不同模态, 随后将它们浅层拼接以完成最终预测。近年来, 受 Transformer 在大语言模型 (LLM) 中成功应用的推动, 将类 Transformer 架构适配到推荐系统已成为热门方向: 一类工作用其建模变长序列特征 (例如 HSTU [27]、Interformer [26]), 另一类工作则用其统一建模不同的特征模态 (例如 OneTrans [31]、HHFT [25]、RankMixer [34])。

然而, 这一趋势忽视了推荐系统与 LLM 之间的一项结构性差异。在 LLM 中, 逐位置的自回归监督要求整个网络始终保留完整的词元级表示, 而计算成本可以分摊到每个词元位置

的预测上。当前推荐系统并无此类要求: 它们针对判别式预测进行训练, 为每个 <用户, 物品> 对生成一组相关性分数, 而不使用词元级自回归损失。因此, 类 Transformer 设计陷入了一种不必要的两难境地: 要么在逐层传播时大幅压缩中间表示, 从而造成信息损失; 要么保留庞大的中间张量, 并随着输入词元数量增长而产生可观的内存与计算开销。这种困境并非推荐任务本身所固有, 而是借用了并不适用的架构前提所导致的结果。

1.2 新范式: 知识库的迭代精炼

我们认为, 判别式推荐更合适的抽象并非序列建模, 而是在紧凑的用户-物品知识库上进行迭代精炼。在这种形式下, 模型维护一个固定大小的 <用户, 物品> 相关性表示, 并通过选择性查询原始多模态证据逐步精炼该表示, 无需实体化尺寸随输入序列长度增长的庞大中间张量。

基于这一原则, 我们提出面向推荐的个性化排序架构 **SlimPer**。如图 1 所示, SlimPer 将个性化排序表述为在统一、固定大小的 <用户, 物品> 知识库上进行的迭代精炼过程。每一层执行三个步骤: (1) 选择: 以当前知识库为条件, 利用具备全模态感知能力的 QKV 注意力查询完整的原始用户侧词元集合; (2) 匹配: 计算检索证据与多方面模板之间显式的点积相关性分数; (3) 精炼: 利用匹配到的证据更新知识库。

这一设计从根源上化解了 Transformer 的两难问题: 通过增加模型深度来提升对 <用户, 物品> 对的理解时, 不再需要维护尺寸随用户侧输入词元数量增长的庞大中间张量。与此同时, 模型质量仍能得到保证, 因为每一层在精炼知识库时都可以直接访问完整的原始用户侧信息。由此产生的更小中间表示可直接降低计算和内存成本 (即 *Slim*)。此外, 知识库将全部学习容量集中于 <用户, 物品> 相关性; 相比必须维护完整词元级表示的架构, 它能够更有针对性地将资源用于相关性匹配 (即 *Smart*)。

此外, 受 HSTU [27] 中仅请求优化 (request-only optimization, ROO) 处理用户侧序列特征所取得成功的启发, 我们显式地将用户侧特征与非用户侧特征分离, 并将 ROO 扩展到所有用户侧特征模态。这是另一项面向推荐的设计选择, 其动机在于: 无论训练还是推理阶段, 推荐模型通常都需要为同一用户排列多个候选物品。

我们使用最终阶段排序模型, 在 Instagram 的两大主要排序场景 Reels 和 Feed 上开展实验。SlimPer 以更为精简的模型架构广泛超越了以往模型, 同时实现了更低的计算和内存成本。

* 同等贡献。† 项目负责人兼通讯作者 (cxj@meta.com)。

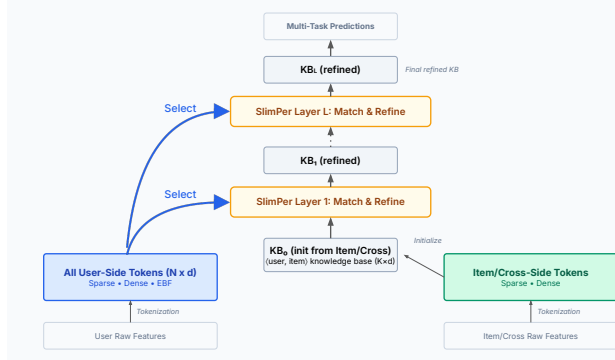


图 1: SlimPer 的高层概览。三项设计原则：(1) 精简 (Slim) – 固定大小的知识库 ($K \times d$)，成本与输入长度 N 解耦；(2) 完整访问 (Complete Access) – 每一层均查询用户侧词元；(3) ROO 感知 (ROO-Aware) – 用户侧词元仅计算一次，并在一次请求中的所有候选物品之间共享。

1.3 主要贡献

综上，我们的主要贡献如下：

- (1) 将迭代精炼作为个性化排序的新范式：我们将判别式推荐表述为在紧凑的 < 用户, 物品 > 知识库上进行的迭代精炼过程，从而摆脱占主导地位的模仿 Transformer 范式。在每个精炼步骤中，模型直接查询完整的原始多模态词元集合，从而将模型深度与输入序列长度解耦。该设计借助固定大小的中间表示实现了每层 $\mathcal{O}(N)$ 的复杂度，有效化解了类 Transformer 架构扩展到长用户历史时所面临的效率-质量权衡。
- (2) 兼具面向推荐效率的统一多模态架构：SlimPer 自然地提供了一个统一骨干网络，仅使用 MLP、QKV 注意力、线性投影和 RMSNorm [30] 等标准神经网络组件，即可处理稀疏、稠密和序列特征。此外，受 HSTU 在用户历史建模中仅请求优化 (ROO) 有效性的启发，SlimPer 将 ROO 感知扩展到所有用户侧特征模态：每个请求批次仅计算一次用户侧词元，并在同一请求内的所有候选物品之间共享，从而减少冗余计算和内存开销。与模仿 Transformer 的基线相比，这些设计选择共同提升了排序质量和系统效率。
- (3) 面向真实推荐系统的内在可解释性：迭代精炼框架还具备内在可解释性。在每一层中，QKV 注意力机制都会将推荐结果与具体的用户交互历史事件直接关联，使模型行为在真实推荐场景中更易于解释和调试。

2 方法

2.1 设计理念

SlimPer 的设计原则源于这样一个观察：推荐任务只需要判别式预测，而不需要逐 token 的自回归输出。第 1 节已经介绍了在紧凑知识库上进行迭代精炼的思想，以及由此带来的架构优势（即深度与长度解耦、完整的信息访问和更加集中的容量分配）。下面我们将这些原则形式化为一个具体架构。

2.2 问题形式化

考虑一个推荐请求，其中包含用户 u 以及待排序的候选物品集合 $\mathcal{I} = \{i_1, i_2, \dots, i_c\}$ 。输入由多种特征模态构成：稀疏特征集合 ($\mathbf{S}$)、稠密特征集合 ($\mathbf{D}$) 以及用户交互历史事件序列 ($\mathbf{E}$)。目标是学习一个评分函数 $f(u, i; \mathbf{S}, \mathbf{D}, \mathbf{E}) \rightarrow \mathbb{R}^r$ ，通过将物品 i 与

表 1: 不同的信息模态

模态	输入类型	代表性信号
稀疏特征	高基数类别 ID	汇总后的用户兴趣（例如互动最多的创作者或类别），提供显著但粒度较粗的偏好信号。
序列特征	结构化用户参与记录	捕获单次用户行为（例如消费的内容、停留时间、参与类型和时间戳）的细粒度行为信号，提供高分辨率的时间信息。
稠密特征	连续数值	上下文及实时信号，例如用户级统计量（如 CTR）、物品质量指标（如转发率）以及上下文属性（如时间或设备）。

用户的潜在兴趣表示进行匹配来估计二者的相关性。在多任务设置中，该函数同时预测代表不同参与度信号的 τ 个目标。

2.3 符号与预备知识

我们将模型限制为一组最精简的构建模块：MLP（采用 SiLU 激活函数的多层感知机）、 $\mathcal{L}$ （分段线性投影： $\beta = \mathcal{L}(\alpha) = \rho\alpha$ ， $\rho \in \mathbb{R}^{L_{\text{out}} \times L_{\text{in}}}$ ），以及全模态 < 用户, 物品 > 知识库 $\mathcal{X}$ ；后者由非用户侧输入 token 初始化为 $\mathcal{X}^0 = \mathcal{L}(\mathbf{S}_{\text{in}})$ 。知识库的形状为 $\mathcal{X}^k \in \mathbb{R}^{K \times d}$ ，其中 K 为知识库槽位数， d 为嵌入维度。通常，各层的知识库大小可以不同；在实验中，我们令所有层均保持 $K = 64$ 。完整的符号说明见附录 B。

2.4 整体架构

SlimPer 模型由多模态 token 化模块以及堆叠在其后的 SlimPer 层组成。整体架构如图 2 所示。模型以原始稀疏特征、稠密特征以及用户历史事件序列为输入。所有特征首先经由各自对应的 tokenizer 进行 token 化（见表 5），生成一组多头的 d 维嵌入。为简化表述，我们在不失一般性的前提下以单头形式描述算法。每一层都执行“选择-匹配-精炼”循环（第 2.6 节），利用标准的查询-键-值（Query-Key-Value, QKV）注意力完成全模态感知的 token 选择和显式相关性匹配。

模型通过显式解耦用户侧与物品侧特征，天然具备仅请求优化（request-only optimization, ROO）感知能力。以往，ROO 往往与质量和效率之间的权衡相关（例如早期排序中的双塔架构），或者仅被狭义地用于用户历史建模。我们的设计将 ROO 感知扩展到整个模型，涵盖用户历史之外的全部用户侧输入特征。对于每个请求批次，所有用户侧特征仅进行一次 token 化；同一请求内的所有物品共享一份用户侧 token 副本，从而最大限度降低内存与计算成本。

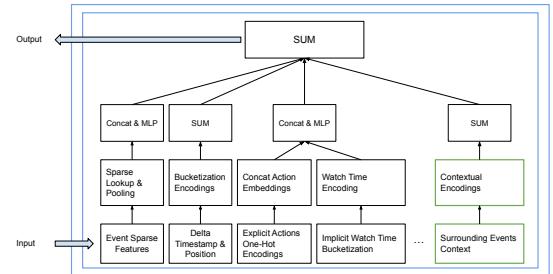


图 3: 事件建模模块，利用内容、参与类型、时间及上下文信息对每个事件进行编码。

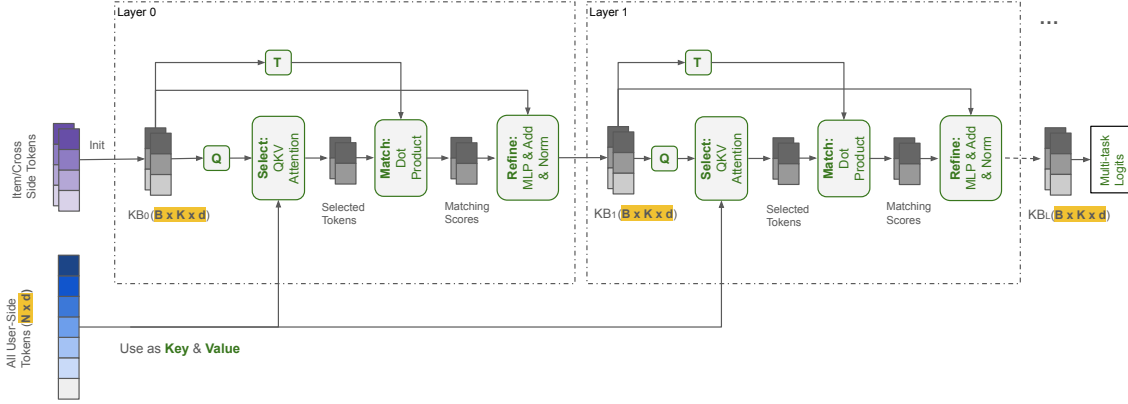


图 2: 多层 SlimPer 模型的架构。每一层通过查询完整的用户侧 token 集合（生产环境中 $N=\mathcal{O}(10^3)-\mathcal{O}(10^4)$ ），对紧凑知识库（ $K=16-64$ 个槽位）进行迭代精炼，从而将堆叠层的内存与计算成本同输入序列长度解耦。

2.5 Token 化模块

在统一骨干网络中，所有原始输入特征都会被转换为模态特定的 token。稀疏特征通过嵌入池化被 token 化为固定大小的向量。每个用户历史事件通过融合内容、参与类型、时间和上下文信息，被编码为一个 $1 \times d$ 的 token。稠密特征则经拼接后通过 MLP 投影。最终得到的多模态 token 集合作为 SlimPer 各层的输入。标准组件（嵌入池化与稠密特征处理）的完整 token 化细节见附录 C。

事件 token 的上下文编码。直观而言，用户行为往往表现出局部连续性：将一个事件与其邻近事件结合起来理解时，该事件包含的信息最为充分。为了捕获这些局部上下文关系，我们引入额外的上下文编码 $\mathbf{e}_{\text{context}}^{(t)}$ ，显式汇总每个事件 token 周围的邻域信息（图 3 中的绿色框）：

$$\mathbf{e}_{\text{context}}^{(t)} = \Psi_{\text{context}}(x_{t-w+1}, \dots, x_t) \quad (1)$$

其中， Ψ_{context} 表示编码函数， w 为窗口大小， x_t 表示从位置 t 处的用户历史事件中收集的信息。我们探索了以卷积神经网络 [11]、滑动窗口注意力 [2] 及二者组合作为 Ψ_{context} ；所有变体均取得了相近的性能。我们还尝试在各事件位置使用不同信息进行上下文编码；详情参见第 4.5.1 节的消融研究。

2.6 SlimPer 层：选择、匹配与精炼

如图 2 所示，每个 SlimPer 层均遵循一个三步迭代精炼过程。在步骤 1 和步骤 2 中，来自所有模态的 token 被联合处理，其中使用 QKV 注意力检索相关的用户侧 token，并计算显式相关性匹配分数。步骤 3 随后利用匹配得到的证据精炼知识库。这一设计将计算与内存资源从用户侧 token 两两之间的交互，重新分配到判别式用户-物品交互的学习上。表 2 比较了 SlimPer 与代表性类 Transformer 架构的计算及内存分配。

2.6.1 步骤 1：通过全模态感知注意力选择 token. 在基于注意力的 token 选择过程中，我们利用所有输入特征模态的信息。令 $\mathcal{X}^k$ 表示第 k 次迭代时的统一知识库，它代表模型当前对用户-物品对的理解。查询向量 $\mathbf{Q}$ 由 $\mathcal{X}^k$ 经分段线性投影得到：

$$\mathbf{Q} = \mathcal{L}(\mathcal{X}^k) \in \mathbb{R}^{q \times d}. \quad (2)$$

直观而言，在 $\mathcal{X}^k$ 所编码的当前用户-物品上下文条件下， $\mathbf{Q}$ 捕获模型希望从用户侧 token 中检索的信息。所有用户侧输入 token 均充当键 $\mathbf{K}$ 和值 $\mathbf{V}$ （可选地进行线性投影）。我们将其组织为两个键值集合：固定大小的稀疏特征和可变长度的序列特征。对两个集合分别应用 QKV 注意力来计算注意力权重，再利用这些权重聚合相应的值，得到注意力输出 $\mathbf{R}$ ：

$$\mathbf{R} = \Phi(\mathbf{Q}, \mathbf{K}) \mathbf{V} \quad (3)$$

其中，注意力核 Φ 取决于模态。对于固定大小的稀疏 token，由于 token 数量恒定，我们采用由 MLP 实现的轻量参数化核。对于可变长度的用户历史序列特征，我们采用无参数的标准缩放点积核 [22]。稠密特征通常维度较小（例如约 1000 个浮点数），因此不经过注意力核，而是在每一层直接传递并融合：

$$\Phi(\mathbf{Q}, \mathbf{K}) = \begin{cases} \text{MLP}(\text{Concat}(\mathcal{L}(\mathbf{Q}), \mathcal{L}(\mathbf{K}))), & \text{稀疏特征} \\ \text{Softmax}(\gamma \mathbf{Q} \mathbf{K}^T), & \text{序列特征} \end{cases} \quad (4)$$

其中， γ 表示 softmax 温度。将这一形式应用于两条用户侧 token 流，可以得到两个基于注意力的表示：

$$\mathbf{R}_s = \Phi_s(\mathbf{Q}, \mathbf{S}) \mathbf{S}, \quad \mathbf{R}_s \in \mathbb{R}^{q \times d}, \quad (5)$$

$$\mathbf{R}_e = \Phi_e(\mathbf{Q}, \mathbf{E}) \mathbf{E}, \quad \mathbf{R}_e \in \mathbb{R}^{q \times d}. \quad (6)$$

注意力输出 $\mathbf{R}_s$ 和 $\mathbf{R}_e$ 汇总了从两条 token 流中选出的证据。在我们的实现中，原始稀疏特征 token $\mathbf{S}$ 和序列特征 token $\mathbf{E}$ 直接用作 QKV 注意力中的键和值。我们也探索了通过额外线性投影生成键和值，但未观察到性能提升；因此，为提高计算效率，我们省略了这些投影。

这一设计形成了全模态感知的注意力机制。由于查询表示 $\mathbf{Q}$ 源自统一知识库 $\mathcal{X}^k$ ，它融合了所有输入模态的信息。因此，稀疏特征信息可以影响序列特征上的 token 选择，反之亦然。

2.6.2 步骤 2：显式多维点积相关性匹配. 为执行相关性匹配，我们通过线性变换，从统一的 $\langle \text{用户}, \text{物品} \rangle$ 知识库中生成一组多维模板 $\mathbf{T} \in \mathbb{R}^{t \times d}$ ：

$$\mathbf{T} = \mathcal{L}(\mathcal{X}^k), \quad \mathbf{T} \in \mathbb{R}^{t \times d}. \quad (7)$$

相关性匹配分数通过模板与各条流的注意力输出之间的显式点积计算：

表 2: 各架构的单物品大 O 复杂度。最右列为每层的用户-物品交互容量, 第 2.7 节分析了 SlimPer 的 $K \times$ 优势。典型生产环境取值: $N = \mathcal{O}(10^3) - \mathcal{O}(10^4)$, $K = 64$, $q = 16$, $B \approx 5$ (训练) / $\mathcal{O}(10^2)$ (推理)。

模型	Token 化成本 (单物品)	堆叠层成本 (单物品)	用户-物品交互容量
传统 DLRM 模型 • 不具备 ROO 感知; 不支持序列特征	内存: $\mathcal{O}(N)$ 计算: $\mathcal{O}(N)$	内存: $\mathcal{O}(L \cdot 64)$ 计算: $\mathcal{O}(L \cdot N \cdot 16)$	$\mathcal{O}(L \cdot N \cdot 64)$
ROO 感知的类 Transformer 模型 (如 HSTU) • 面向用户历史建模的 ROO 感知	内存: $\mathcal{O}(N/B)$ 计算: $\mathcal{O}(N/B)$	内存: $\mathcal{O}(L \cdot N/B)$ 计算: $\mathcal{O}(L \cdot (N+1)^2/B)$	$\mathcal{O}(L \cdot N)$
非 ROO 感知的类 Transformer 模型 (如 OneTrans) • 不具备 ROO 感知	内存: $\mathcal{O}(N)$ 计算: $\mathcal{O}(N)$	内存: $\mathcal{O}(L \cdot N)$ 计算: $\mathcal{O}(L \cdot N^2)$	$\mathcal{O}(L \cdot N)$
次二次复杂度的类 Transformer 模型 (如线性注意力或滑动窗口注意力) • 线性化注意力	内存: $\mathcal{O}(N)$ 计算: $\mathcal{O}(N)$	内存: $\mathcal{O}(L \cdot N)$ 计算: $\mathcal{O}(L \cdot N)$	$\mathcal{O}(L \cdot N)$
SlimPer • 对所有特征模态均具备 ROO 感知	内存: $\mathcal{O}(N/B)$ 计算: $\mathcal{O}(N/B)$	内存: $\mathcal{O}(L \cdot K)$ 计算: $\mathcal{O}(L \cdot N \cdot q)$	$\mathcal{O}(L \cdot N \cdot K)$

$$\lambda_s = \text{DotProduct}(\mathbf{R}_s, \mathbf{T}), \quad \lambda_s \in \mathbb{R}^{q \times t}, \quad (8)$$

$$\lambda_e = \text{DotProduct}(\mathbf{R}_e, \mathbf{T}), \quad \lambda_e \in \mathbb{R}^{q \times t}. \quad (9)$$

2.6.3 步骤 3: 精炼知识库。随后, 对相关性分数进行归一化并通过 MLP_μ 变换, 以精炼 < 用户, 物品 > 知识库 $\mathcal{X}^k$, 并生成下一层表示 $\mathcal{X}^{k+1}$ 。更新知识库时, 还会拼接稠密 token $\mathbf{D}$ 以融入稠密特征信息:

$$\mathcal{X}^{k+1} = \mathcal{X}^k + \text{MLP}_\mu \left(\text{Concat}(\text{RMSNorm}(\lambda_s), \text{RMSNorm}(\lambda_e), \mathcal{L}(\mathcal{X}^k), \mathbf{D}) \right) \quad (10)$$

在每一层, 我们可以选择为每个预测任务输出一个任务嵌入, 表示模型当前对该任务的理解。可以先将各层的任务嵌入求和, 再应用最终的线性投影来生成任务 logit:

$$\mathbf{p}_p^k = \text{MLP}_p(\mathcal{L}(\mathcal{X}^k)) \quad (11)$$

其中, $p \in [1, \tau]$ 为预测任务的索引。

2.7 通过堆叠 SlimPer 层实现迭代个性化

SlimPer 天然支持堆叠: 多个 SlimPer 层按顺序组合, 每一层都在共享知识库 $\mathcal{X}$ 上执行一次“选择-匹配-精炼”循环 (第 2.6 节)。在生产设置中, 我们通常堆叠 5 到 10 层。

其直觉在于, 更深的层能够受益于先前层累积的证据, 从而逐步实现信息更充分的个性化。例如, 较早的层可能专注于收集与目标物品有关的宽泛证据, 如用户过去是否与狗狗视频发生过互动。这些累积证据随后为后续层提供条件, 使其能够关注更加细微的信号, 例如近期行为 (如频繁跳过狗狗视频) 是否表明用户兴趣已经发生变化。通过这种方式, 较后的层可以判断应强化还是削弱较早识别出的偏好。

这种迭代精炼能力由两项架构特性实现。第一, 每一层都直接关注所有模态的完整原始输入 token 集合, 既保留了细粒度信号, 又避免了维护大型中间张量所带来的内存开销。类 Transformer 架构会逐步将信息压缩到中间隐藏状态中, 而 SlimPer 在每个深度都执行无损的跨模态与跨层 (用户 $\leftrightarrow$ 物品) 融合。第二, 在整个精炼过程中, 知识库始终保持固定大小。因此, 增加深度并不要求中间内存随输入长度增长; 即使用户具有很长的交互历史, 也可以切实地堆叠许多精炼层。

2.7.1 信息论依据。实现这些效率收益的固定大小知识库看似可能会牺牲信息, 但分析表明情况恰恰相反: SlimPer 的瓶颈并未丢弃容量, 而是将其重新导向与任务相关的信号。

瓶颈具有充裕的容量。推荐任务对每个 < 用户, 物品 > 对只需要给出 τ 个标量预测, 因此目标 Y 与输入 X 之间的互信息上界为 $H(Y) \leq \tau$ 比特, 远低于 $K \times d$ 知识库的表示容量。此外, 如表 2 的交互容量列所示, SlimPer 实现了 $\mathcal{O}(L \cdot N \cdot K)$ 的用户-物品交互容量, 与 DLRM 相当, 并且是仅达到 $\mathcal{O}(L \cdot N)$ 的类 Transformer 模型的 $K \times$ 。产生这一差异的原因是, Transformer 将其 $\mathcal{O}(N^2)$ 计算中的大部分分配给了用户侧 token 的两两交互, 而这些交互并不直接服务于相关性估计; SlimPer 则将每层全部 $K \times N$ 次交互用于知识库到 token 的相关性匹配。因此, 该瓶颈约束的是中间内存, 而不是用户-物品相关性的学习容量。值得注意的是, 无论注意力是二次的还是线性化的, 这一容量论证都成立。线性注意力 [10] 和滑动窗口注意力 [2] 等次二次变体将每层计算降低到 $\mathcal{O}(N)$, 但它们仍然: (i) 跨深度传递大小为 N 的 token 级中间张量, 使内存保持在 $\mathcal{O}(L \cdot N)$ 而非 $\mathcal{O}(L \cdot K)$; (ii) 将交互容量用于用户侧 token 混合, 使用户-物品交互容量停留在 $\mathcal{O}(L \cdot N)$, 比 SlimPer 低 K 倍。因此, 我们预期 SlimPer 相较于高效注意力基线不仅具有效率优势, 也能保持质量优势, 因为这种收益源于容量的分配方式, 而不是注意力渐近阶数的降低。

迭代访问可防止不可逆的信息损失。标准瓶颈架构受数据处理不等式约束: 对于马尔可夫链 $X \rightarrow T \rightarrow Y$, 压缩过程中丢弃的信息无法恢复。SlimPer 打破了这条链: 每一层都直接对原始输入 token X 进行交叉注意力, 而不只是访问上一层的压缩表示。用信息瓶颈框架 [21] 的术语来说, SlimPer 执行的是迭代式压缩: 每一层都提取额外的任务相关互信息 $I(\mathcal{X}^k; Y)$, 同时原始证据始终可供访问, 以纠正较早阶段的遗漏。这一特性得到了附录 E 中注意力热力图的验证: 随着层数增加, 模型的关注点会从宽泛的历史上下文逐步收窄到相关性最高的近期信号。

3 相关工作

我们从三条研究脉络来定位 SlimPer: 面向固定大小输入的特征交互学习、面向变长事件序列的用户历史建模, 以及来自相邻领域的交叉注意力瓶颈架构。

3.1 特征交互学习

这一范式专注于捕获固定大小输入特征之间复杂的高阶交互。因子分解机 (Factorization Machines) [17] 和 DeepFM [5] 通过

成对因子分解引入了高效的二阶交互建模。DLRM [14] 将稀疏嵌入、经 MLP 处理的稠密特征与显式点积交互相结合，这一设计此后得到了广泛引用。深度交叉网络（Deep Cross Network, DCN）[23] 和 DCNv2 [24] 提出使用交叉网络对次数有界的特征交叉进行建模，而 AutoInt [19] 则在特征嵌入上应用多头自注意力，以学习隐式高阶交互。近期工作更加重视架构的可扩展性：DHEN [29] 和 Wukong [28] 系统研究了交互模块的堆叠，并展示了固定大小特征上的良好缩放定律。

与 *SlimPer* 的关系。这些方法将用户历史视为预先聚合的特征向量。*SlimPer* 通过知识库细化涵盖了它们的能力，同时还使每一层都能直接访问单个用户历史事件。

3.2 用户历史建模

随着用户历史数据日益丰富，有效捕获序列特征中的信号变得愈发重要。DIN [33] 和 DIEN [32] 引入了面向目标的用户交互历史注意力来建模用户意图。基于 Transformer 的模型（BST [4]、BERT4Rec [20]、SASRec [9]）将用户事件视为词元并应用完整自注意力，虽然能够获得富有表现力的用户表征，却会产生限制其扩展的 $\mathcal{O}(N^2)$ 开销。

针对超长用户历史的研究探索了多种策略：(1) 基于检索的方法：SIM [16] 使用两阶段硬搜索来检索相关行为；(2) 基于记忆的方法：MIMN [15] 维护长期兴趣的固定大小表征；(3) 基于压缩的方法：LONGER [3] 采用词元合并，HSTU [27] 使用随机长度采样；(4) 离线聚合：TWIN-V2 [18] 和 DV365 [13] 将长程建模转移到预计算嵌入上。这些策略有一个共同模式：它们通过限制可用于用户-物品交互建模的原始行为细节数量来降低长历史的处理成本，即以牺牲对用户历史信息细粒度、目标感知的访问能力，换取大规模场景下的可处理性。

尽管取得了这些进展，用户历史建模通常仍是一个独立组件，其输出经过汇总后再通过后期融合与固定大小特征结合，从而限制了深层跨模态交互。近期的统一架构（如 OneTrans [31]、HHFT [25]、RankMixer [34]）使用 Transformer 风格的设计联合处理所有模态，但在扩展到长序列时也继承了相同的二次复杂度开销。

与 *SlimPer* 的关系。*SlimPer* 通过每层 $\mathcal{O}(K \cdot N)$ 的计算量以及不随序列长度增长的固定大小内存占用解决了这一权衡，使模型能够扩展至 10k+ 个事件，同时在每一层访问原始用户侧词元且不造成信息损失。

3.3 交叉注意力瓶颈架构

SlimPer 使用固定大小知识库迭代地对更大词元集合执行交叉注意力，这在结构上与其他领域的架构存在相似之处。Perceiver [8] 和 Perceiver IO [7] 维护一个可学习的潜变量数组，并让其迭代地对原始输入执行交叉注意力，从而实现关于输入长度的线性复杂度。Set Transformer [12] 针对集合结构数据引入了具有类似瓶颈的诱导集合注意力块（Induced Set Attention Blocks, ISAB）。Flamingo [1] 将语言模型到视觉词元的交叉注意力用于多模态理解。

尽管 *SlimPer* 与这些方法都遵循通过紧凑瓶颈使计算与输入长度解耦的原则，但它引入了通用架构所不具备的若干推荐场景特有归纳偏置：(1) 判别式相关性聚焦：知识库由物品侧特征初始化，并完全用于 < 用户, 物品 > 相关性评分；(2) 显式相关性匹配：专门的点积步骤（步骤 2）计算所检索证据与多面模板之间的分数，这在 Perceiver 风格设计中没有对应机制；(3) ROO 感知：用户侧词元仅计算一次，并在一次请求内的所有候选之间共享，从而减少冗余计算和内存用量；(4) 模

态相关的注意力核：*SlimPer* 对固定大小的稀疏词元采用参数化的基于 MLP 的注意力，对变长序列特征则采用缩放点积注意力。

4 实验

4.1 实验设置

数据集. 我们在 Instagram 的末阶段排序模型上开展实验，使用与当前模型训练时相同的训练与评估数据。

评估指标. 我们使用归一化熵（Normalized Entropy, NE）评估模型性能。NE 是一种标准离群指标，其变化方向与 A/B 测试结果一致。NE 将模型的二元交叉熵损失（BCEloss）除以一个朴素基线的 BCEloss 进行归一化；该基线对所有样本均预测全局标签均值 p_k 。经验上，约 0.03% 的 NE 改善被认为具有显著意义。

$$NE_k = \frac{BCEloss(\hat{y}^{(k)}, y^{(k)})}{-[p_k \ln(p_k) + (1 - p_k) \ln(1 - p_k)]}, \quad (12)$$

基线. 我们将 *SlimPer* 与 Instagram 现有模型进行比较。现有模型采用后期融合架构，将用于建模变长用户交互历史序列的 HSTU [27] 与用于建模固定大小稀疏/稠密特征的 Wukong [28] 拼接起来。这些模型负责 Instagram 主要产品界面的末阶段排序，构成了一个成熟的基线。为确保公平比较，我们使用相同的训练与评估日期范围，并报告相对于该基线的 NE 改善。除非另有说明，所有其他实验设置（包括优化器配置和特征集合）在各模型间均保持不变。

4.2 离线性能

我们在 Reels 和 Feed 上对 *SlimPer* 与基线进行了离线评估，主要结果汇总于表 3。在两个产品界面上，我们均观察到以下三点：(1) *SlimPer* 在所有任务上都取得了稳定的 NE 改善。受篇幅限制，我们仅报告两个产品界面的代表性任务。(2) 在相同序列长度下，相较于基线，*SlimPer* 同时取得 NE 和 QPS 优势，表明所提出的设计在提高预测质量的同时也提升了效率。(3) 我们观察到了缩放趋势：随着序列长度增加，NE 收益也随之提升。例如，在 Feed 的转发预测任务上，NE 改善从 1k 序列长度时的 -0.49% 增长至 4k 序列长度时的 -0.94%，相当于模型质量的相对收益接近 2×，但 QPS 下降了 16%。

这些实证结果与表 2 中的大 O 分析一致：*SlimPer* 关于 N 的线性计算复杂度及其 $\mathcal{O}(L \cdot K \cdot d)$ 内存占用，直接解释了相较于二次复杂度基线所观察到的 11%+ QPS 提升和 9-18% 内存降低。由于这种成本降低来自使用固定大小知识库细化替代完整序列上的自注意力，因此它可以推广至任何通过二次复杂度自注意力处理完整词元序列的类 Transformer 方法。

4.3 A/B 测试性能

我们在 Instagram Reels 上发布了使用 5k 个事件的 *SlimPer*，并在 Instagram Feed 上发布了使用 4k 个事件的 *SlimPer*，即第 4.2 节所述模型。在完成预测测试和回溯测试后，*SlimPer* 在两个产品界面上均推向全量流量，并在多个主要用户参与指标上带来了具有统计显著性的提升；其汇总后的顶线影响约为一次典型统计显著发布的 10×，这是 2025 年建模变更所带来的显著改进。此外，*SlimPer* 未导致生态系统护栏指标（例如完整性、多样性和时效性）退化，并使训练与推理 GPU 容量维持在近似中性水平，证实离线观察到的效率提升（第 4.2 节）能够转化为真实在线服务收益。

表 3: SlimPer 与基线的比较。NE 和 QPS 为相对变化 (%)；内存为平均用量的绝对变化 (%)；均相对于各产品界面特定的基线。NE/内存越低越好，QPS 越高越好。

产品界面	模型配置	NE (%)						QPS (%)	内存差异 (%)
		转发	跳过	点赞	评论	关注	收藏		
Reels	基线, 2k 个事件	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	SlimPer, 2k 个事件	-0.51	-0.31	-0.23	-0.41	-0.35	-0.35	+11.0	-9.32
	SlimPer, 5k 个事件	-0.80	-0.57	-0.49	-0.64	-0.53	-0.62	-10.0	+4.59
Feed	基线, 1k 个事件	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	SlimPer, 1k 个事件	-0.49	-0.28	-0.54	-0.52	-0.71	-0.62	+12.5	-18.12
	SlimPer, 4k 个事件	-0.94	-0.52	-0.91	-0.94	-1.00	-1.05	-16.07	+2.04

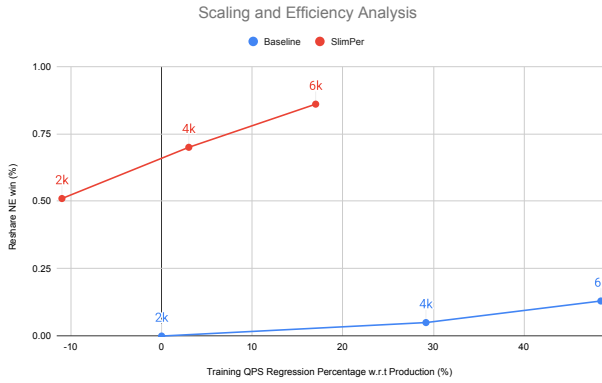


图 4: 随用户历史序列长度增长 (2k-6k) 的质量-效率缩放关系。每条曲线对应一个模型 (SlimPer 或基线) 在不同序列长度下的结果；x 轴为训练 QPS 降幅，y 轴为 NE 收益，二者均为相对于基线的百分比 (%)。SlimPer 能以更小的 QPS 降幅维持更大的 NE 收益，而且其优势随序列长度增加而扩大。

2026 年，我们又发布了更多 SlimPer 变体，通过增加模型容量扩展至 10k+ 个事件。这些发布在 Meta 的多个产品界面上带来了显著的用户参与度收益，进一步验证了 SlimPer 在真实约束下的有效性与可扩展性。

4.4 缩放与效率分析

表 2 中的大 O 分析预测，SlimPer 每层 $\mathcal{O}(K \cdot N)$ 的复杂度应当相较于二次复杂度基线带来明显的效率优势，而且这一差距会随序列长度增加而扩大。我们以 Reels 模型为代表，通过在不同序列长度配置下比较 SlimPer 与基线（后期融合的 HSTU + Wukong），对这一预测进行实证验证。基线可用的历史长度受到序列上二次复杂度注意力开销的限制，而 SlimPer 则避开了这一瓶颈。结果汇总于图 4。

主要发现。在同为 2k 的长度下，SlimPer 在转发任务上以 -0.51% NE 超越基线，同时实现更高的吞吐量 (+11% QPS) 和更低的内存用量，即同时提升了质量与效率。随着序列长度增加，这一收益进一步累积：在 6k 长度下，SlimPer 的 NE 达到 -0.86%，而 QPS 降幅不到 20%；相比之下，基线仅取得 -0.13% 的 NE 收益，QPS 却下降了 48%。如曲线所示，随着紧凑知识库吸收更丰富的长期信号，SlimPer 的 NE 单调改善；而采用二次复杂度的基线在更长序列上则呈现收益递减。

这些结果证实，SlimPer 的质量提升并非源于投入更多计算，而是源于将计算更有效地分配给用户-物品相关性匹配。

与表 2 及第 2.7 节的信息论分析一致，SlimPer 每层 $\mathcal{O}(K \cdot N)$ 的成本在实践中转化为少 8-25× 的 FLOPs (附录 F)，而且这一优势随输入长度超线性增长：紧凑知识库将交互容量从主导类 Transformer 模型的 $\mathcal{O}(N^2)$ 用户侧到用户侧计算，转移到每层 $\mathcal{O}(K \times N)$ 的相关性聚焦交互上。

内存效率。在相同序列长度下观察到的 9-18% 内存降低来自两种互补机制：固定大小知识库取代了每一层中大小为 $\mathcal{O}(N)$ 的中间张量，而 ROO 感知设计则让一次请求内的所有候选共享全部用户侧词元的单份副本。

4.5 消融研究

我们通过消融实验验证各项设计决策。

4.5.1 上下文编码的影响. 我们通过比较三种策略，消融研究将周围事件信息融入每个历史词元是否能够提升性能：(1) 无上下文（独立编码），(2) 事件类型上下文（编码周围动作，例如点赞、转发、长时间观看），以及 (3) 完整上下文（联合编码事件类型与事件内容）。引入上下文编码在转发任务上带来了约 -0.15% 的 NE 改善。值得注意的是，这一收益主要来自仅编码事件类型；加入事件内容仅带来边际上的额外收益 (NE < 0.02%)。这表明，用户的行为模式——即他们采取了什么动作——比其交互的具体物品更具信息量，凸显出推荐建模与语言建模之间的重要差异：后者的语义由内容依赖关系决定。

4.5.2 知识库大小与模型深度的影响. 我们进一步消融了知识库大小 K 和堆叠的 SlimPer 层数。知识库大小 $K=64$ 能够在质量与效率之间取得良好平衡；将其缩小至 $K=4$ 虽使 QPS 提升 +23.2%，却导致 NE 退化 +1.2%，证实知识库确实发挥了有效信息瓶颈的作用。就模型深度而言，7 层已能取得优异性能；增加至 9 层时收益仍在延续 (NE -0.12%)，且成本增幅适中。完整消融表见附录 D。

4.6 模型预测的可解释性

SlimPer 的注意力机制可以将推荐结果直接归因于特定的用户历史事件。对于给定请求，模型会生成候选物品与序列词元之间的注意力分数，使实践者能够识别出对每次预测影响最大的前 k 个历史交互。抽样检查表明，注意力权重与排序输出相一致，说明其匹配保真度较高。我们在附录 E 中给出了模型各层注意力模式的详细分析。

包括超参数设置在内的可复现性细节见附录 A。

5 局限性

与近期统一架构的直接比较有限。我们将 SlimPer 与成熟基线 (HSTU + Wukong) 进行比较，而没有直接对比近期的

Transformer 风格统一架构（如 OneTrans、HHFT、RankMixer）。在实践中，独立的统一 HSTU 表现不及这一混合模型；后者针对不同模态采用专门组件（用户历史使用 HSTU，固定大小特征使用 Wukong）。因此，相较任何单一统一架构，该混合模型是更强的比较基准；SlimPer 在明显优于它的同时，还将系统简化为单一骨干网络。不同组织在训练数据、特征集合及服务约束方面的差异，也进一步使其他系统报告的数值无法直接比较。尽管如此，我们的复杂度分析（表 2）具有普适性：任何在完整序列上使用二次自注意力的架构都会面临相同的 $\mathcal{O}(N^2)$ 计算瓶颈；任何维护大小为 N 的中间张量，或为每个候选物品复制用户侧表示的架构，也都会面临相同的内存瓶颈。SlimPer 采用固定大小且具备 ROO 感知能力的知识库，同时解决了这两个问题。

内部数据上的可复现性。所有实验均在 Instagram 内部训练和评估数据上开展，这限制了对我们精确结果的独立复现。此外，我们使用公开基准中难以获得的数据规模将模型训练至收敛，这进一步限制了结果的精确复现，但也验证了 SlimPer 在实际部署中的有效性。为缓解这一问题，我们提供了详细的架构超参数（附录 A），仅使用标准构建模块（MLP、QKV 注意力、线性投影、RMSNorm）描述模型，并在受控实验条件下报告了相对于明确定义基线的提升。我们相信，这些细节足以使实践者在自己的推荐数据集上重新实现并验证该方法。

6 结论

本文提出了 SlimPer，这是一种统一的个性化模型，它将推荐排序重新表述为对紧凑 < 用户, 物品 > 知识库的迭代细化。SlimPer 摒弃了近期工作中占主导地位的模仿 Transformer 范式，实现了独特的效率-质量权衡：模型深度与输入序列长度解耦；原始多模态证据未经压缩地保留下来，并在每一层被直接访问；整个模型容量均聚焦于相关性估计，而非维护词元级表征。SlimPer 还通过消除各层中大小为 N 的中间张量，并利用仅请求优化（request-only optimization）在一次请求内的所有候选之间仅维护一份共享的用户侧词元，进一步降低了 GPU 内存用量。

我们在工业推荐系统上的实验验证了 SlimPer 的有效性。与基于 Transformer 的替代方案相比，所提出的架构以更低的计算成本取得了优异的离线性能，并在受控 A/B 实验中带来了具有统计显著性的用户参与度提升。此外，SlimPer 天然具备可解释性：每个细化阶段的注意力权重揭示了证据如何被选择和聚合，从而有助于进行有原则的调试、审计和模型分析。

展望未来，我们认为有若干值得探索的方向。首先，迭代细化范式可以自然地从事列式用户历史扩展到不存在内在顺序的证据源，例如跨产品界面的参与信号或上游筛选系统生成的候选。由于知识库直接与原始词元交互，而不依赖序列结构，因此 SlimPer 无需改变架构即可整合此类信号。其次，SlimPer 的全模型 ROO 感知设计支持跨排序阶段的高效部署，例如在早期排序阶段，更大的候选集合会进一步放大摊销计算带来的收益。最后，当前的匹配与细化模块有意采用了轻量设计，依赖简单的点积匹配和基于 MLP 的更新。在这些组件中探索表达能力更强的算子，可能在保留每层 $\mathcal{O}(N)$ 复杂度和固定大小内存占用的迭代细化框架的同时，带来进一步的质量提升。更广泛地说，我们相信，围绕个性化场景独特的计算与建模需求所开展的推荐中心式架构设计，将继续推动工业推荐系统取得实际进展。

Acknowledgments

工程协作者. 我们感谢 Meta 各团队的众多协作者，他们在 SlimPer 于 Meta 应用家族中的构建与扩展过程中给予了宝贵合作：Jay Wang, Keming Zhang, Chen Chen, Qichao Que, Wanli Ma, Maxwell Lin-He, Yu Liu, Yanhong Wu, Xinye Zheng, Wen-Yun Yang, Rui Zhang, Jiyan Yang, Zichao Li, Lei Nie, Yan Sun, Rongcheng Lin, Ivan Ji, Cory Nezin, Jeffrey Cordero, Lambert Chu, Leon Yan, Keyi Wu, Yiming Liao, Irra Na, Neel Pawar, Tingting Zhang, Xue Zou, Ziwei Li, Zhe (Joe) Wang, Haifeng Lan, Qinjie Lin, Ye Wang, Huasha Zhao, Jianming He, William Pei, Dawei Fan, Yezhou Huang, Minhan Li, Zhiwei Zhang, Dev (Devashish) Shankar, Jackie (Jiaqi) Xu, Chunzhi Yang, Haoxun Luo, Hao Lin, Prashasti Baid, Lu Zhang, Liang Guo, and Rui Jian.

领导支持. 我们还感谢 Meta 应用家族及 MRS 的领导团队所提供的支持：Jayant Subramanian, Haotian Wu, Jing Qian, Hui Zhang, Shilin Ding, Sophia (Xueyao) Liang, Xinyao Hu, Linhong Zhu, Yisong Song, Shujian Bu, Aameek Singh, Hong Yan, Meihong Wang, Mahesh Srinivasan, Adam (Yang) Song, Nipun Mathur, Lars Backstrom, Jeff Huang, Tessa Lyons-Laing, Deepak Agarwal, and Max Eulenstein. 正是他们的指导与投入使本项工作得以实现。

A 可复现性

表 4 汇总了 SlimPer 在 Feed 和 Reels 上的主要架构超参数。所有模型均采用 Shampoo 优化器 [6]、以 0.05 的学习率训练一个轮次；所有模型配置（包括 SlimPer 和基线）使用完全相同的优化器设置。

表 4: SlimPer 在 Feed 和 Reels 上的架构超参数。

超参数	Feed	Reels
SlimPer 层数 (L)	5	7
知识库大小 (K)	64	64
隐藏维度 (d)	256	256
查询槽位数 (q)	16	16
模板槽位数 (t)	32	32

B 符号与预备知识

为降低架构复杂度并简化实现，我们将模型限制为一组最小化的基础组件；这些组件具有相同的内部结构，仅超参数设置不同。下面定义整个网络中使用的符号和算子模块。

多层感知机 (MLP). MLP 表示标准多层感知机，即由多层全连接层堆叠而成，并在层间应用非线性激活。在实验中，我们使用 SiLU 作为激活函数。

分段线性投影 ($\mathcal{L}$). $\mathcal{L}$ 是分段线性投影模块，它使用可学习投影矩阵 ρ ，将输入张量 $\alpha \in \mathbb{R}^{L_{in} \times d}$ 转换为输出张量 $\beta \in \mathbb{R}^{L_{out} \times d}$ ：

$$\beta = \mathcal{L}(\alpha) = \rho \alpha, \quad \rho \in \mathbb{R}^{L_{out} \times L_{in}}. \quad (13)$$

在网络中，我们多次使用该模块，并且每次使用都具有独立的可学习参数。为简化表示，后续公式省略用于区分各模块的上下标。

全模态感知的 < 用户, 物品 > 知识库 ($\mathcal{X}$). 我们使用 $\mathcal{X}$ 表示全模态 < 用户, 物品 > 知识库。该知识库在第 0 层初始化为：

表 5: 不同信息模式的连续词元化方法。

模式	连续词元化方法
稀疏	嵌入池化 (E-P): 对嵌入进行求和或均值池化, 将稀疏特征词元化, 得到一个或少量固定大小的向量 (例如 $1 \times d$)。
序列	事件建模 (E-M): 聚合并融合描述每个事件的全部信息, 形成固定长度的事件表示 (例如每个事件为 $1 \times d$)。
稠密	稠密处理 (D-P): 对稠密特征进行归一化, 并通过 MLP 将其投影为固定大小的向量 (例如 1024 维)。

$$\mathcal{X}^0 = \mathcal{L}(\mathbf{S}_{\text{in}}) \quad (14)$$

其中, $\mathbf{S}_{\text{in}}$ 表示非用户侧输入词元, 通常来源于物品侧或交叉侧稀疏特征。 $\mathcal{X}^k$ 的形状可以随层而变化, 但在实验中, 所有层均使用一致的 $K \times d$ 形状 (其中 $K = 64$)。

C 词元化细节

在统一骨干网络中, 所有原始输入特征首先被转换为一组模式专属词元。该步骤既保留了不同特征类型之间的语义差异, 也为后续共享交互堆栈做好准备。我们考虑三类输入模式: 稀疏特征、用户历史序列特征和稠密特征。相应的词元化方法汇总于表 5。

C.0.1 多模式词元化.

嵌入池化模块 (E-P). 首先通过标准嵌入查找表将稀疏特征映射为嵌入向量。对于具有多个取值的特征 (例如用户最喜欢的前 k 个类别), 我们应用池化操作:

$$\mathbf{S}^j = \text{Pool}(\{\mathbf{e}_{j,1}, \mathbf{e}_{j,2}, \dots, \mathbf{e}_{j,k}\}) \quad (15)$$

其中, $\text{Pool}(\cdot)$ 可以是均值池化或求和池化。将所有稀疏特征的池化结果拼接后, 得到表示稀疏特征词元的张量 $\mathbf{S} = (\mathbf{S}^{(1)}, \dots, \mathbf{S}^{(|\mathcal{S}|)}) \in \mathbb{R}^{|\mathcal{S}| \times d}$, 其中 d 为每个词元的维度。

事件建模模块 (E-M). E-M 模块将用户历史中的每个事件处理为词元表示。每个事件通常由若干稀疏特征描述, 这些特征分别表示事件内容、用户的显式行为类型、隐式观看时长、互动时间戳以及其他元数据。我们使用 $\mathbf{E} = (\mathbf{E}^{(1)}, \dots, \mathbf{E}^{(t)}, \dots)$ 表示词元化后的用户历史事件序列。为得到每个 $\mathbf{E}^{(t)}$, E-M 模块构建若干并行编码组件, 将每个组件投影到共享嵌入维度 d , 再通过求和将其组合:

$$\mathbf{E}^{(t)} = \text{MLP}_E^s(\mathbf{e}_{\text{sparse}}^{(t)}) + \mathbf{e}_{\text{temporal}}^{(t)} + \text{MLP}_E^e(\mathbf{e}_{\text{type}}^{(t)}) + \mathbf{e}_{\text{context}}^{(t)} \quad (16)$$

其中, $\mathbf{e}_{\text{sparse}}^{(t)}$ 是由描述事件物品的池化稀疏特征拼接而成的内容嵌入; $\mathbf{e}_{\text{type}}^{(t)}$ 是同时捕获显式和隐式行为的事件类型编码; $\mathbf{e}_{\text{temporal}}^{(t)}$ 是时间编码, 由位置编码与时间差编码相加得到。

稠密处理模块 (D-P). 稠密特征经过拼接, 并通过一个简单的 MLP 投影为固定大小的向量:

$$\mathbf{D} = \text{MLP}(\text{Concat}(\mathbf{d}_1, \mathbf{d}_2, \dots, \mathbf{d}_{|\mathcal{D}|})) \quad (17)$$

D 消融实验

我们通过消融实验验证 SlimPer 迭代精炼范式背后的设计选择。基线对应表 3 中使用的包含 5k 个事件的 **Reels SlimPer** 模型, 其知识库大小为 $K = 64$, 包含 7 个精炼层。

D.0.1 知识库大小的影响. 表 6 给出了知识库槽位数 K (即 $K \in \{4, 8, 16, 32, 64\}$) 的消融实验。直接评估迭代精炼瓶颈的信息容量。基线配置为 $K = 64$ 。对于每种配置, 查询大小 q 定义为知识库大小的四分之一, 模板大小 t 则定义为 K 的二分之一。

表 6: 知识库大小的消融实验 (相对于 $K = 64$ 的基线)。

K, q, t	NE (%)	QPS (%)	内存差异 (%)
(64, 16, 32)	0.0	0.0	0.0
(4, 1, 2)	+1.2	+23.2	-9.53
(8, 2, 4)	+0.53	+21.2	-8.19
(16, 4, 8)	+0.36	+17.8	-7.43
(32, 8, 16)	+0.18	+4.25	-1.31

D.0.2 精炼层数的影响. 表 7 给出了 SlimPer 堆叠精炼层数 (即 $L \in \{3, 5, 7, 9\}$) 的消融实验。该实验评估迭代精炼是否能够持续提升排序质量, 并考察性能增益从何种深度开始趋于饱和。基线配置包含 7 个精炼层。

表 7: SlimPer 精炼层数的消融实验 (相对于 7 层基线)。

层数	NE (%)	QPS (%)	内存差异 (%)
7	0.0	0.0	0.0
3	+0.32	+21	-8.60
5	+0.14	+8.8	-5.50
9	-0.12	-7.16	+4.92

E 模型预测的可解释性

SlimPer 的迭代精炼设计具备内在可解释性: 每一层的 QKV 注意力会将用户过去的交互与当前预测直接联系起来, 因此能够将推荐结果归因于具体的历史事件。不同于将不透明组件拼接起来的以往模型, SlimPer 的注意力分数自然地解释了为何向某个用户推荐某个物品。

跨层注意力分析. 对于给定请求, 模型会生成一个注意力矩阵, 将候选物品 (查询) 映射到序列词元 (键)。我们汇总 200 个序列长度为 5k 的请求上的注意力分布, 并将其可视化热力图; 其中横轴表示按序排列的序列索引, 纵轴表示候选物品。

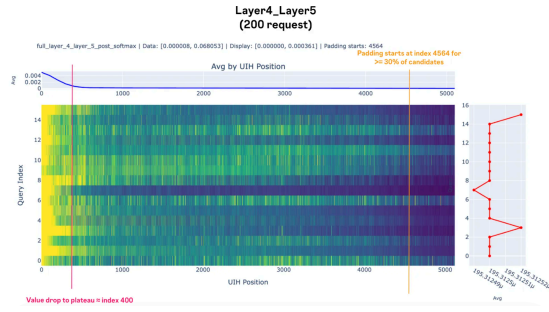


图 6: SlimPer 高层的注意力热力图。注意力进一步收窄至最近的 ~400 个事件。

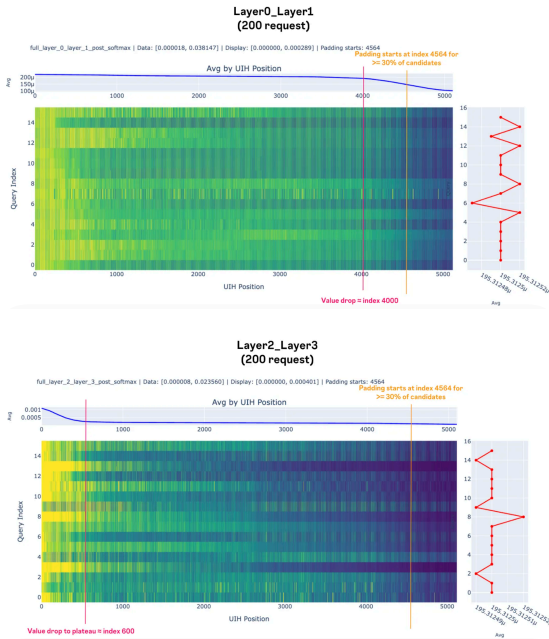


图 5: SlimPer 底层和中间层的注意力热力图。底层广泛关注完整历史，中间层则集中于较近期的事件。

如图 5 所示，在最底层，模型广泛关注前 4k 个序列索引，从而捕获粗粒度的历史上下文。在中间层，注意力集中于前几百个索引，聚焦于范围更窄、时间更近的一组交互。

在高层（图 6）中，该模式进一步锐化—注意力集中于最近的 ~400 个事件。这种层次化注意力行为—底层关注广泛上下文，高层逐步收窄关注范围—与 SlimPer 的迭代精炼设计一致：较早的层收集有关用户偏好的广泛证据，而较后的层利用最近且影响最大的行为信号来精炼预测。

F FLOPs 分析

除了吞吐量（QPS）之外，我们还通过统计前向传播中的所有矩阵乘法（不包括嵌入查找、归一化和激活）来量化计算成本。表 8 报告了不同序列长度下估算的 FLOPs。在双方共同采用的默认长度 2k 下，SlimPer 的 FLOPs 约为基线的 1/8。随着序列长度增加，差距进一步扩大：在 6k 长度下，SlimPer (~6 GFLOPs) 的 FLOPs 仍约为基线原生 2k 长度配置 (~24 GFLOPs) 的 1/4，同时实现了 -0.86% 的 NE 改善。

表 8: 每个样本估算的前向传播 FLOPs (GFLOPs)，统计全部矩阵乘法。基线仅统计 HSTU 编码器（依赖序列长度的组件）。

序列长度	基线 (HSTU)	SlimPer	比率
2048	~24	~3	~8×
4096	~74	~5	~16×
6144	~150	~6	~25×

参考文献

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Mariber Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. 2022. Flamingo: a Visual Language Model for Few-Shot Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [2] Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arXiv preprint arXiv:2004.05150* (2020).
- [3] Zheng Chai, Qin Ren, Xijun Xiao, Huizhi Yang, Bo Han, Sijun Zhang, Di Chen, Hui Lu, Wenlin Zhao, Lele Yu, Xionghang Xie, Shiru Ren, Xiang Sun, Yaocheng Tan, Peng Xu, Yuchao Zheng, and Di Wu. 2025. LONGER: Scaling Up Long Sequence Modeling in Industrial Recommenders. *arXiv:2505.04421 [cs.LG]* <https://arxiv.org/abs/2505.04421>
- [4] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior sequence transformer for e-commerce recommendation in alibaba. In *Proceedings of the 1st international workshop on deep learning practice for high-dimensional sparse data*. 1–4.
- [5] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A Factorization-Machine Based Neural Network for CTR Prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*. 1725–1731.
- [6] Vineet Gupta, Tomer Koren, and Yoram Singer. 2018. Shampoo: Preconditioned Stochastic Tensor Optimization. In *International Conference on Machine Learning*. PMLR, 1842–1850.
- [7] Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer, Olivier Hénaff, Matthew M. Botvinick, Andrew Zisserman, Oriol Vinyals, and João Carreira. 2022. Perceiver IO: A General Architecture for Structured Inputs & Outputs. In *International Conference on Learning Representations (ICLR)*.
- [8] Andrew Jaegle, Felix Gimeno, Andrew Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. 2021. Perceiver: General Perception with Iterative Attention. In *International Conference on Machine Learning (ICML)*.
- [9] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [10] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. In *ICML*. 5156–5165.
- [11] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.
- [12] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. 2019. Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks. In *International Conference on Machine Learning (ICML)*.
- [13] Wenhao Lyu, Devashish Tyagi, Yihang Yang, Ziwei Li, Ajay Somani, Karthikeyan Shanmugasundaram, Nikola Andrejevic, Ferdi Adepurta, Curtis Zeng, Arun K. Singh, Maxime Ransan, and Sagar Jain. 2025. DV365: Extremely Long User History Modeling at Instagram. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25)*. 4717–4727.
- [14] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G Azzolini, et al. 2019. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091* (2019).
- [15] Qi Pi, Weijie Bian, Guorui Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Practice on long sequential user behavior modeling for click-through rate prediction. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2671–2679.

- [16] Qi Pi, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, Xiaoqiang Zhu, and Kun Gai. 2020. Search-based user interest modeling with lifelong sequential behavior data for click-through rate prediction. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 2685–2692.
- [17] Steffen Rendle. 2010. Factorization Machines. In *2010 IEEE International Conference on Data Mining*. 995–1000.
- [18] Zihua Si, Lin Guan, ZhongXiang Sun, Xiaoxue Zang, Jing Lu, Yiqun Hui, Xingchao Cao, Zeyu Yang, Yichen Zheng, Dewei Leng, et al. 2024. Twin v2: Scaling ultra-long user behavior sequence modeling for enhanced ctr prediction at kuaishou. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 4890–4897.
- [19] Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. 2019. AutoInt: Automatic Feature Interaction Learning via Self-Attentive Neural Networks. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM '19)*. 1161–1170.
- [20] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.
- [21] Naftali Tishby, Fernando C Pereira, and William Bialek. 2000. The Information Bottleneck Method. *arXiv preprint physics/0004057* (2000).
- [22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*. 5998–6008. <https://arxiv.org/abs/1706.03762>
- [23] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & Cross Network for Ad Click Predictions. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 12:1–12:7.
- [24] Ruoxi Wang, Rakesh Shivanna, Derek Zhiyuan Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed H. Chi. 2021. DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems. In *Proceedings of The Web Conference 2021*. 1785–1797.
- [25] Liren Yu, Wenming Zhang, Silu Zhou, Tao Zhang, Zhixuan Zhang, and Dan Ou. 2025. HHFT: Hierarchical Heterogeneous Feature Transformer for Recommendation Systems. *arXiv preprint arXiv:2511.20235* (2025).
- [26] Zhichen Zeng, Xiaolong Liu, Mengyue Hang, Xiaoyi Liu, Qinghai Zhou, Chaofei Yang, Yiqun Liu, Yichen Ruan, Laming Chen, Yuxin Chen, Yujia Hao, Jiaqi Xu, Jade Nie, Xi Liu, Buyun Zhang, Wei Wen, Siyang Yuan, Kai Wang, Wen-Yen Chen, Yiping Han, Huayu Li, Chunzhi Yang, Bo Long, Philip S. Yu, Hanghang Tong, and Jiyan Yang. 2025. InterFormer: Towards Effective Heterogeneous Interaction Learning for Click-Through Rate Prediction. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25)*. 6225–6233.
- [27] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *Proceedings of the 41st International Conference on Machine Learning*. 58484–58509.
- [28] Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Shen Li, Yanli Zhao, Yuchen Hao, Yantao Yao, Ellie Dingqiao Wen, Jongsoo Park, Maxim Naumov, and Wenlin Chen. 2024. Wukong: Towards a Scaling Law for Large-Scale Recommendation. In *Proceedings of the 41st International Conference on Machine Learning*. 59421–59434.
- [29] Buyun Zhang, Liang Luo, Xi Liu, Jay Li, Zeliang Chen, Weilin Zhang, Xiaohan Wei, Yuchen Hao, Michael Tsang, Wenjun Wang, et al. 2022. DHEN: A deep and hierarchical ensemble network for large-scale click-through rate prediction. *arXiv preprint arXiv:2203.11014* (2022).
- [30] Biao Zhang and Rico Sennrich. 2019. Root Mean Square Layer Normalization. *arXiv preprint arXiv:1910.07467* (2019).
- [31] Zhaoqi Zhang, Haolei Pei, Jun Guo, Tianyu Wang, Yufei Feng, Hui Sun, Shaowei Liu, and Aixin Sun. 2025. OneTrans: Unified Feature Interaction and Sequence Modeling with One Transformer in Industrial Recommender. *arXiv preprint arXiv:2510.26104* (2025).
- [32] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep interest evolution network for click-through rate prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 5941–5948.
- [33] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.
- [34] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, et al. 2025. Rankmixer: Scaling up ranking models in industrial recommenders. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6309–6316.